Module-IX

Lecture-I

Introduction to Automatic Test Pattern Generation (ATPG) and ATPG Algebras

1. Introduction

In the last module we saw that in a circuit some faults are "easy to test" and some others are "difficult to test". Further, we have also seen an algorithm that can estimate the "easy to test faults" and the ones that are "difficult to test". Fault simulation algorithms are used to determine test patters for easy to test faults and for the others, sensitization–propagation -justification approach is used. In the last module we also discussed in detail, most of the commonly used fault simulation algorithms. In this module we will study automatic test pattern generation (ATPG) using sensitization–propagation -justification approach. We will first introduce the basics of ATPG and ATPG algebras. Following that in the next lecture of the module we will present D-algorithm--the basic and first ATPG algorithm developed. Also, we will present basics of some other advanced ATPG algorithms, which were successors of the D-algorithm.

2. What is ATPG

ATPG stands for automatic test pattern generation. As the name suggests, ATPG procedure involves generation of input patterns that can ascertain presence or absence of fault(s) at some location(s) in a circuit. There are several ways of ATPG. In the last module we have seen two of them- (i) fault simulation and (ii) sensitization–propagation –justification. These techniques are based on Boolean logic manipulations and are most widely used. However, it must not be felt that only logic based schemes can be used for ATPG. Here we briefly discuss another interesting ATPG technique that requires thermal

imaging technique [1]. Thermal imaging technique involves taking images of the silicon (of the circuit) and then drawing conclusions based on temperature profile of various regions of the silicon. In case of ATPG based on imaging technique, one needs to find an input pattern that results in difference of temperature at a location (which is being tested) with and without fault. For example, when a net say A, has stuck-at-0 fault and primary inputs are applied such that net A gets 1, then temperature in A will be abnormally higher, due to high short circuit current. The advantage is, we need not have to worry about propagating the effect of the fault to primary outputs, as all internal points are observable in terms of temperature profile. So, sensitization is enough to generate a test pattern. However, the instrumentation cost for thermal imaging is high. So, widely used ATPG algorithms are based on logic manipulations. Henceforth, when we talk about ATPG it will imply techniques based on logic operations. Next we will see basics of ATPG algebra, which will be required to understand D-algorithm.

3. ATPG algebra

As we all know, Boolean algebra comprising of 0 and 1, is used for studying behavior of digital circuits. However, if we consider circuits with fault, then Boolean algebra will not suffice to represent signal values. Let us take an example. Consider the AND gates given in Figure 1. In order to represent that I1=1,I2=1 detects s-a-0 fault at the output, we need to mark O1 as 1/0-indcating that under normal condition O1 is 1 and under fault it is 0. Similarly, in case of Figure 1(b) we need to mark O1 as 0/1--indicating that under normal condition O1 is 0 and under fault it is 1. So we can see that a single bit (0 or 1) is not enough for representing and studying circuits with fault.

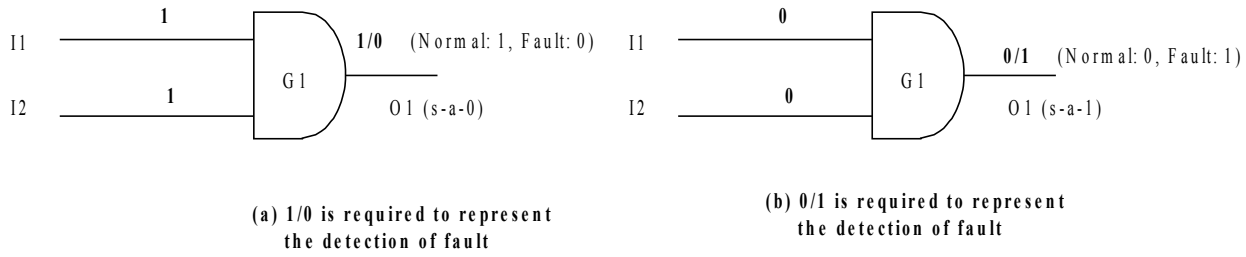


Figure 1. Requirement of more than Boolean (0/1) algebra for studying fault behaviors

So, the algebra suitable for studying circuits with fault should have higher-order notations than Boolean algebra. In other words, we need more notations than just 0 and 1 for representing both the "good" and the "failing" circuit simultaneously. Roth [2] proposed a five valued algebra which suitable for ATPG based on sensitize-propagate-justify approach. Table 1 illustrates Roth's 5-valued algebra.

Table 1. Roth's five value algebra

Symbol	Implication	Normal Circuit	Faulty Circuit
0	(0/0)	0	0
1	(1/1)	1	1
X	(X/X)	X	X
D	(1/0)	1	0
$\overline{D}$	(0/1)	0	1

It may be noted that along with 0 and 1, three more notations namely, D, $\overline{D}$ and X are used. D is used to represent 1/0—i.e., under normal condition the net has value 1 and 0 under presence of a fault. Similarly, $\overline{D}$ represents 0/1. X is used to represent the case when the signal value of a net is unknown. The use of more notations allows determination of signal values for both normal and faulty circuits in one pass. Since a test pattern requires that a difference be maintained between the two circuits, it is computationally fastest to represent both circuits in the algebra, rather than one of them at a time. Figure 2 illustrates two input AND gates where the input/outputs are marked using notations from Roth's 5-valued algebra.

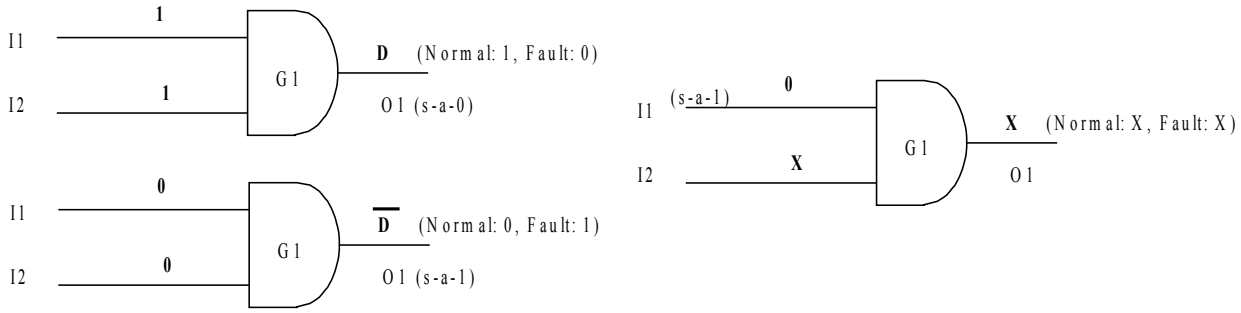


Figure 2. Example to illustrate Roth's five value algebra

Logical operations using Roth's 5-valued algebra is very similar to Boolean algebra, only we need two Boolean operations when D or $\overline{D}$ is one of the inputs. Some examples are as follows

- 1 AND D = D; 1 AND 1/0 involves two computations—(i) 1 AND 1=1, (ii) 1 AND 0=0.
- 0 OR $\overline{D}$ = $\overline{D}$; 0 OR 0/1 involves two computations—(i) 0 OR 0=0, (ii) 0 OR 1=1.
- $\mathrm{NOT}(\overline{D})$=D; NOT(0/1) involves two computations—(i) NOT(0)=1, (ii) NOT(1)=0.

When one input is X in logical operation then output is also X, if others inputs are non controlling; else output is determined by the controlling input. Some examples are given below

- X AND 1 =X; 1 is non controlling in AND logic
- X AND 0 =0; 0 is controlling in AND logic

4. Types of ATPG algorithms

4.1 Exhaustive

Testing by "exhaustive" technique implies applying all input combinations and checking for "functional" correctness. As already discussed in last module, for an n-input circuit, an exhaustive test technique would require 2^n input combinations. Considering the time required for exhaustive testing, it is infeasible for circuits with more than about 15 inputs. So exhaustive testing techniques are not used in ATPG.

4.2 Random

As elaborated in the last module, random ATPG basically involves three steps:

- Generate a random pattern
- Determine how many faults are detected by the random pattern (fault simulation)
- Continue the above two steps till no (or few) new faults are detected by the random pattern.

Random patterns can detect "easy to test faults" and the algorithm being fast, it is applied at initial phases of ATPG. Also discussed before, heart of ATPG by random patterns are fault simulation algorithms. We saw four major fault simulations algorithms in the last module namely, sequential, parallel, concurrent and deductive. However, for the faults which cannot be detected by random patterns, we need deterministic ARPG techniques. Now we will see two such types of algorithms (i) symbolic difference and (ii) path sensitization.

4.3 Symbolic Difference

ATPG by symbolic difference is basically based on well-known Shannon's expansion theorem. Let us explain this by an example. Consider an arbitrary Boolean function as $f(x_1, x_2, ..x_n)$. By Shannon's expansion rule, the function can be expanded about any variable x_1 say, as $f(x_1, x_2, ..x_n) = \overline{x_1}.f(0, x_2, ..x_n) + x_1.f(1, x_2, ..x_n)$.

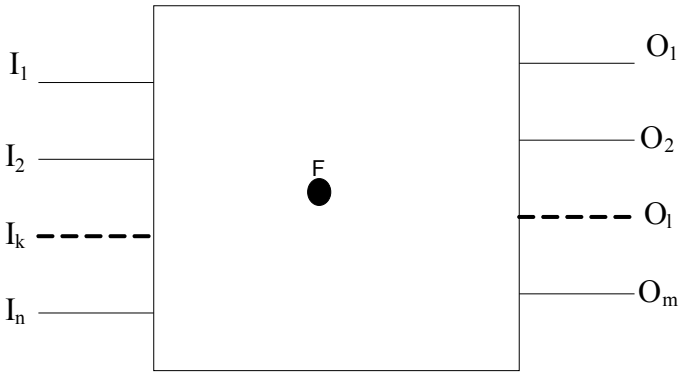


Figure 3. Block diagram of a circuit with a fault at point F

Figure 3 illustrates block diagram of a circuit with *n* inputs, *m* outputs and a fault at net F. Let the following Boolean functions hold

- $g = f_F(I_1, I_2, ..I_n)$ be the function for the Boolean value at net F (in terms of all the inputs)
- $o_i = f_i(g, I_1, I_2, ..I_n),\ 1 \le i \le m$ be the functions for output lines O_1 to O_m, under failure at point F. It may be noted that without fault all the output lines are functions of the inputs. With the failure, the output lines are functions of the inputs and the Boolean value at point F.

Now, Boolean difference, or Boolean partial derivative, of the circuit block output $f_i, 1 \le i \le m$ with respect to *g* is:

$$\frac{\partial f_i}{\partial g} = f_i(1, I_1, I_2, ..I_n) \oplus f_i(0, I_1, I_2, ..I_n)$$

For detecting s-a-0 fault at location F, we need

1. $g = f_F(I_1, I_2, ..I_n) = 1$
2. $(\exists j, 1 \le j \le m) \wedge \left(\frac{\partial f_j}{\partial g} = f_j(1, I_1, I_2, ..I_n) \oplus f_j(0, I_1, I_2, ..I_n) = 1 \right)$

The equations mean the following.

The first equation requires that to test a stuck-at-0 fault at F, 1 should be applied at F (sensitization). Second equation says that in order to detect the fault, there is at least one output such that the Boolean difference of the output with respect to F must be 1 (i.e., the output must change when the signal at F changes from 1 to 0.)

Similarly, for detecting s-a-1 fault at location F, we need

1. $g = f_F(I_1, I_2, ..I_n) = 0$
2. $(\exists j, 1 \le j \le m) \wedge \left(\frac{\partial f_j}{\partial g} = f_i(1, I_1, I_2, ..I_n) \oplus f_i(0, I_1, I_2, ..I_n) = 1 \right)$

Unfortunately, due to high complexity the Boolean difference is not an efficient way to compute test patterns for large circuits.

4.4 **Path sensitization**

As discussed in the last module, ATPG by path sensitization method is generally applied for "difficult to test faults" and comprises three phases.

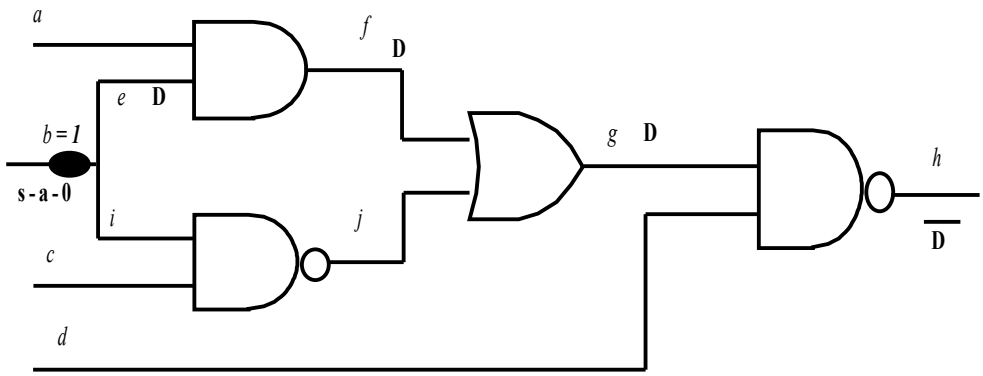
1. **Fault sensitization:** In this step a stuck-at fault is activated by setting the signal driving the faulty net to an opposite value from the fault value.
2. **Fault propagation**: In this step a path is selected from the fault site to some primary output, where the effect of the fault can be observed for its detection.
3. **Line justification**: In this step the signals in (internal) nets or some primary inputs, which were assigned for fault sensitization/propagation, are justified by setting (remaining) primary inputs of the circuit.

In the second and third steps, a conflict may occur, where a necessary signal assignment contradicts some previously-made assignment. When conflicts occur we need to take a new alternative path for fault propagation and see if all signals can be justified.

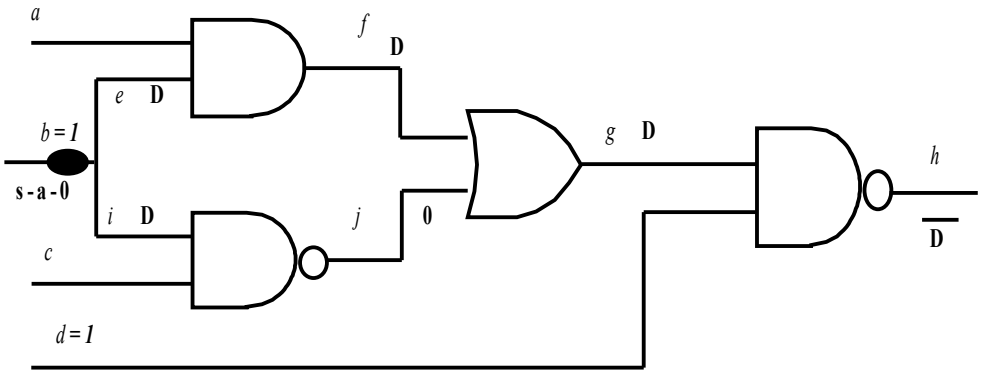
We have seen some simple examples for ATPG using the path sensitization (sensitization-propagation-justification) approach in the last module. Now we will see a bit more complex example of ATPG using the path sensitization approach. However, instead of using Boolean algebra (as in last module), we will use Roth's 5 valued algebra. Following that, in the next lecture we will learn D-algorithm--the primary and formal algorithm for ATPG using path sensitization.

As shown in Figure 4 (a), there is a s-a-0 fault in input *b*. To sensitize the fault, simply *b* is to be made 1. Now let us take the path "***e-f-g-h***" for propagating the effect to the output *h*. The signals in the nets of the path, in terms of Roth's 5 valued algebra, are shown in Figure 4(a). It may be noted that we have successfully

propagated $\overline{D}$ to the output; it implies that the fault can be propagated to the output using the path selected. Now let us justify the signals, by setting the inputs of the gates in the path selected (for fault propagation), but not themselves being in the path, to non-controlling values. For example, net *j*, is a input to the OR gate that is in the path selected for fault propagation, but *j* is not itself in the path; so *j* is to be 0. Similarly, *d* is to be 1, and *a* is to be 1. However, it must be noted that *j* cannot be made 0; if *c*=1 then *j*=D and if *c*=0 then *j*=1. So we have reached a conflict at *j;* Figure 4(b). Now we must backtrack and select a new path for propagation.



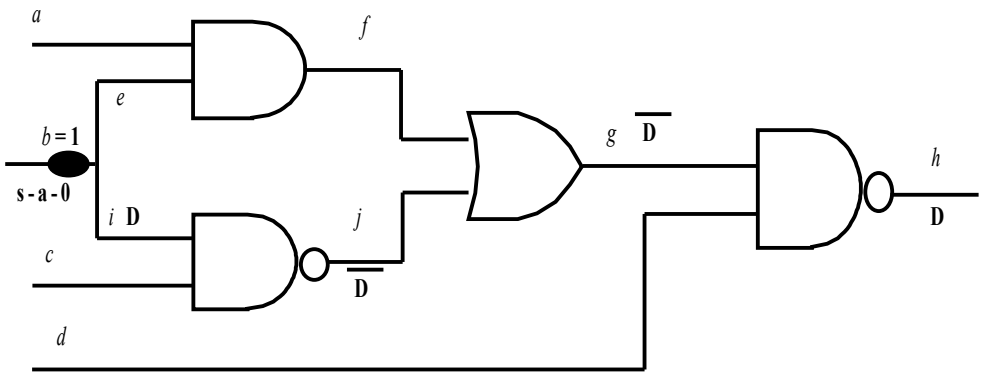
(a) Sensitization by *b*=1
Propagation by path e-f-g-h



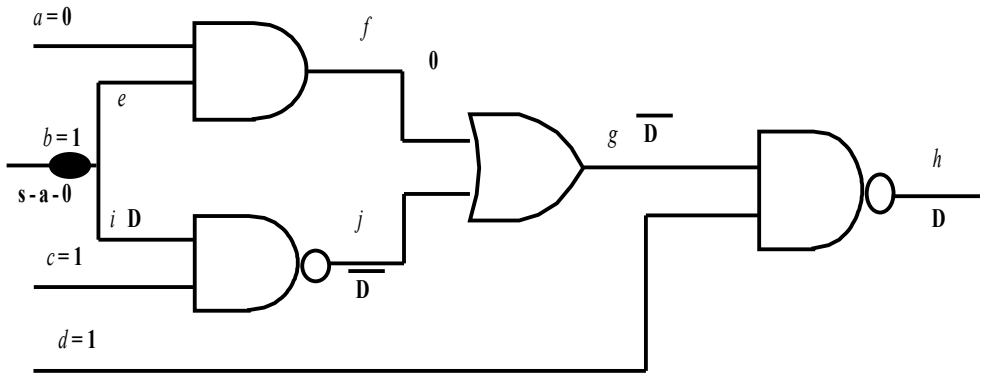
(b) Justification j=0
j*=D, *if c=1 and j=1 if c=0
(conflict at j)

Figure 4. Illustration of backtrack in path sensitization based ATPG

Figure 5 shows the ATPG if the new path is "***i-j-g-h***". Figure 5 (a) shows the values in the nets required to propagate the fault to the output. In the new path we have again successfully propagated the fault to the output, but now a D is obtained instead of $\overline{D}$ (as in the previous case}. This implies that the fault can be propagated to the output using the new path, however, the reflection of the effect is reversed. To justify the signals, we require d=1, c=1, f=0. f=0 is easily obtained by setting a=0. So we have successfully, justified the signals if the path is "***i-j-g-h***"; test pattern is a=0,b=1,c=1, d=1 and effect at output is D.



(a) Sensitization by b=1
Propagation by path i-j-g-h



(b) Justification d=1,c=1,f=0,a=0
(Successful)

Figure 5. Successfully found test pattern in an alternative path (circuit of Figure 4)

5. **Conclusions**

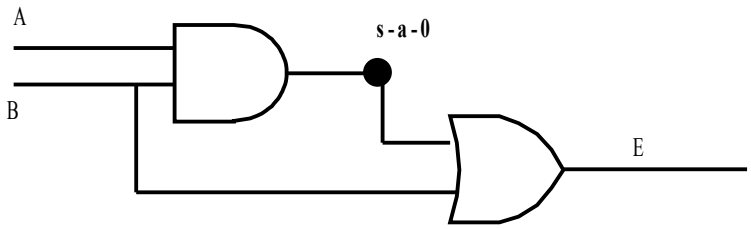
In this lecture we have discussed the basic types of ATPG algorithms and a new 5 valued algebra required in all the major ATPG algorithms (based on path sensitization). In the next lecture we will study in detail the D-algorithm, which is the basis of all path sensitization based ATPG algorithms. As a point of motivation, it must be stated that most of the modern and widely accepted “deterministic” ATPG algorithms are based on path sensitization approach and roots drawn from D-algorithm.

References

1. W. Claeys, S. Dilhaire, and V. Quintard, "Laser probing of thermal behavior of electronic components and its application in quality and reliability testing", Microelectronic Engineering, vol. 24, pp. 441 - 420, 1994. :Elsevier
2. J. P. Roth, W. G. Bouricius, and P. R. Schneider, “Programmed Algorithms to Compute Tests to Detect and Distinguish Between Failures in Logic Circuits,” IEEE Trans. on Electronic Computers, vol. EC-16, no. 5, pp. 567–580, Oct. 1967.

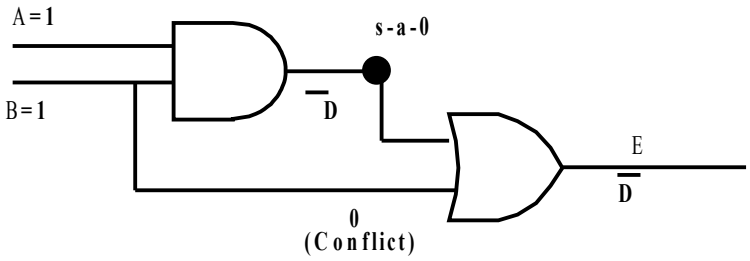
Questions and Answers

1. Using path sensitization and Routh's algebra generate test pattern for the s-a-0 fault in the circuit given below:



Answer:

To sensitize the fault, 1 is to be applied at the output of the AND gate. Also, there is only one path for fault propagation-- fault location to E. Now, to justify we need to apply a 0 at the second input of the OR gate. This leads to a collision. For justification we need a 0 in the second input of the OR gate and to sensitize the fault B is to be 1, which makes the second input of the OR gate a 1. This is illustrated in the figure below in terms of Routh's algebra.



Now, since there is no other alternative path for fault propagation, this fault is not testable. In fact the answer is true. This fault is not testable. This example actually illustrates another utility of ATPG algorithm—finding redundancy in circuits. If we observer carefuuly, function represented by this circuit is: E=A.B+B. This can be simplified as B(A+1), which is B. So these two gates are redundant. So, if ATPG by path sensitization discovers that a fault is not testable, then there is redundancy in the circuit.